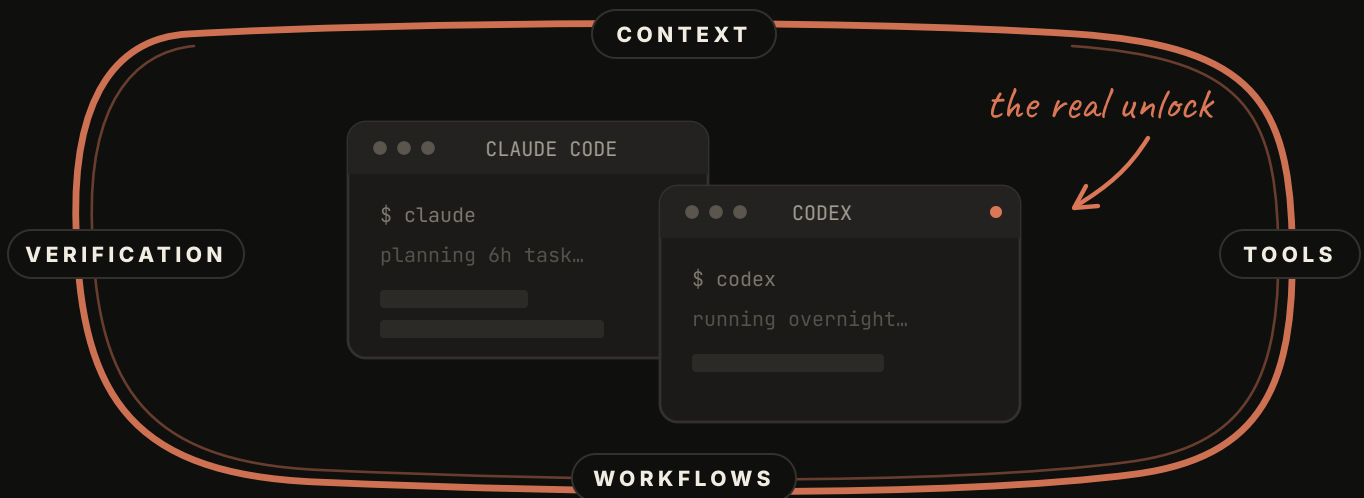


THE COMPANION PLAYBOOK TO THE VIDEO

The Orchestration Layer Playbook

*The four-pillar system that lets Claude Code and Codex run for hours without you — **context, tools, workflows, and verification** — with the skills, snippets, and rollout plan to build it.*

**PATRICK ELLIS**

CTO & CO-FOUNDER · AI-NATIVE STARTUP

read this before your next long run →

The bottleneck moved.

Why this playbook exists, what's inside, and how to turn it into working infrastructure this month.

For Fable and Astra class models, build around long-horizon autonomous work. The shift from short coding exchanges to sustained, end-to-end tasks calls for a corresponding change in your workflow: clearer context, useful tools, repeatable processes, and reliable verification.

The 10x, 20x, 100x leverage is **not coming from the models getting smarter**. It's coming from the orchestration layer around them: the context, tools, workflows, and verification that let codegen agents run for hours without you. Teams investing in that layer are compounding; teams that aren't are quietly falling behind.

THE THESIS

"The bottleneck is no longer with the model. It's with the orchestration layer."

This playbook distills the full video into a reference you can build from: the mental model, every workflow shown on screen, the real skill files behind them, and a 30-day rollout. Timestamps like [04:57] point back to the moment in the video.

// HOW TO USE IT

- **Skim the framework** (p.4), then score your setup against each pillar's checklist.
- **Steal the snippets**. Blocks marked **TEMPLATE** are meant to be copied and adapted; the `review-and-push` skill on p.9 is the real file I run.
- **Run the 30-day rollout** (p.15) — one pillar per week. Don't try to build all four in a weekend.
- Sections marked **FIELD NOTES** are patterns from my planning notes that were cut from the final edit for time.

// CONTENTS

- 03 The paradigm shift
- 04 The four-pillar framework ★
- 05 Pillar 1 — Context
- 06 Pillar 2 — Tools
- 07 Infrastructure as context (+ permissions)
- 08 Pillar 3 — Workflows & SOPs
- 09 Inside my review-and-push skill ★
- 10 Pillar 4 — Verification
- 11 Case study — analytics platform in a week
- 12 Scaling to teams of agents
- 13 Field notes — cut from the edit
- 14 Why this compounds
- 15 The 30-day rollout
- 16 Every tool & link in one place

WHO THIS IS FOR

AI-native engineers and founders who already run Claude Code or Codex daily — and want them running for six hours straight, not six minutes between questions.

goal: 100 questions → 5

Both labs are building the same thing: agents that run without you.

Anthropic and OpenAI are converging on long-horizon autonomy — give the model everything up front, let it run until it's done. [01:51]

// THE CONVERGENCE

Greg Brockman laid out the "AGI-pilled" Codex vision on the OpenAI podcast: their first CLI tool was deemed *too iterative*, so they rebuilt Codex around cloud tasks that run for hours — eventually days — and come back **completely done**. [02:24]

Claude Code grew up as a hands-on pair programmer for messy, real-world codebases. The broader lesson applies to Fable and Astra class models: equip the agent for both exploration and sustained execution, then verify the result against the same quality bar. [03:51]

CODEx · THE
ACADEMIC

Systematic, precise, one-shots the plan

Brilliant *when you hand it a full spec*

Weaker at figuring things out mid-flight

CLAUDE CODE · THE
SCRAPPER

A million bash searches, greps its way around

Thrives in messy, unclear codebases

Iterates, asks, adapts on the fly

OLD RULE → Codex when you knew the spec; Claude Code when you didn't. NEW REALITY → set either up properly, and let it run.

// THE ONBOARDING ANALOGY

Treat the model like an **incredibly capable college student on day one**: brand new to your team, your company, your project, and you. Explicitly outline the resources it has, your expectations around code quality, and the tools it can reach — anticipate every question you'd get while onboarding a person. [04:26]

Nobody's model ships with your context baked in. It lives in your head, in outdated docs, in Slack. The orchestration layer is how you onboard the model in **seconds instead of weeks** — every single session.

TACTIC № 1 — DO THIS ON EVERY MODEL RELEASE
[05:00]

Stop testing new models with single prompts. Hand them the most ambitious end-to-end project you can think of — a raw meeting transcript plus mockups, turned into sophisticated PRDs against your company wiki. That's exactly how the analytics case study on p.11 started. The new capability lives in multi-step tasks; single prompts can't reveal it.

2024

PAIR PROGRAMMER
SHORT LOOPS ·
HANDHOLDING



2026

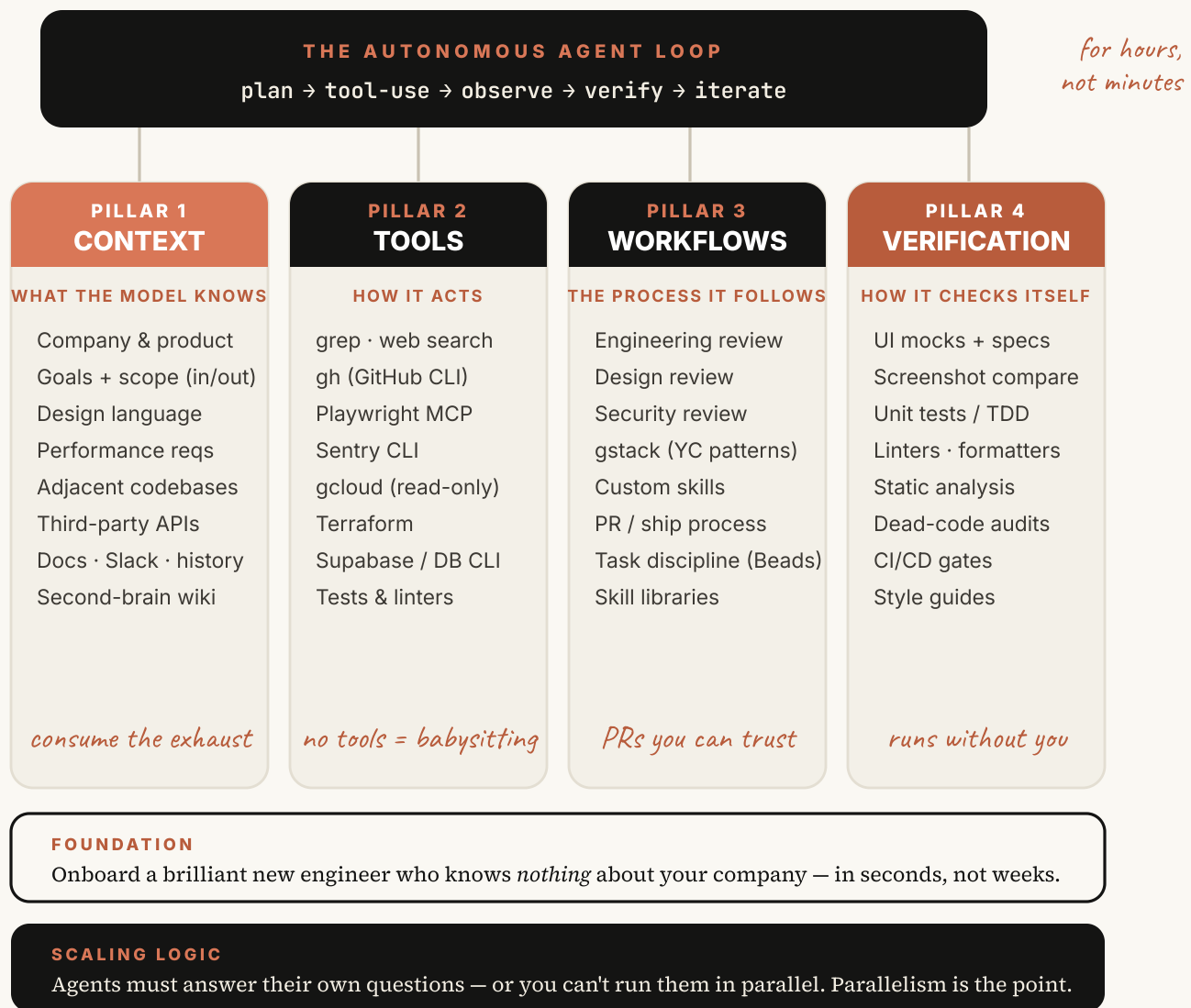
AUTONOMOUS WORKER
HOURS OF UNATTENDED
WORK

ABOUT THE COMPLAINTS

The people on X saying the new models "over-ask" or go off the rails aren't lying about their experience — they're running **2026 models on a 2024 workflow**. The model didn't get worse. The bar for your setup moved. If it's asking too many questions, it wasn't given enough answers up front.

The four-pillar orchestration layer

Everything you set up around the model so it can answer its own questions. Four pillars holding up one autonomous loop. [05:30]



I used to describe this layer as three pillars — context, tools, verification. The fourth, **workflows / standard operating procedures**, is the newest addition and the one almost nobody builds: the packaged process the model follows, not just the facts it knows. [05:34]

Each pillar gets its own page next — with a checklist to score yourself against and a snippet to steal. The case study on p.11 shows all four running end-to-end on real production infrastructure.

Feed it everything a new hire would ask for.

Any information about your company and product the model would otherwise have to ask you about — captured once, reused by every session. [05:52]

// THE CONTEXT INVENTORY

- ☐ **Company & product** — what you're building, for whom, and why it wins
- ☐ **Goals & scope** — what's explicitly in *and* out of the current push
- ☐ **Design language** — the aesthetic, components, and tone you prefer
- ☐ **Performance requirements** — budgets, SLOs, scale expectations
- ☐ **Adjacent codebases** — microservice-style sibling repos it should know exist
- ☐ **Third-party APIs** — the contracts you integrate against
- ☐ **Docs, Slack, history** — where decisions and tribal knowledge live

TACTIC — CONSUME THE EXHAUST

Your best context already exists as a by-product of working: meeting transcripts, task boards, Slack threads, git history, your second-brain wiki. Don't write context from scratch — **pipe the exhaust in**. The analytics platform on p.11 was kicked off from a raw meeting transcript plus mockups. [05:00]

Why it pays off: planning with full context up front — the first 10% of the work — feels like a pain, but it's what lets you skip hundreds of mid-run questions later. That's the whole trade. [18:17]

// A CLAUDE.MD SKELETON TEMPLATE

```
CLAUDE.md · keep it under ~200 lines

# {Product} — agent onboarding

## What this is
{2-3 sentences: product, customer, the job this
repo does. Written for a brilliant stranger.}

## Scope right now
IN: {current initiative, e.g. analytics v2}
OUT: {explicitly not now — stops scope creep}

## Architecture in one breath
{monorepo layout, key services, data flow.
  Adjacent repos: ../api, ../gallery, ../infra}

## Design language
{tokens/components to reuse; "extend, don't
  invent"; link the style-guide skill}

## Quality bar
- typecheck: pnpm verify:quick  tests: pnpm test
- lint must pass; no dead code; no new deps
  without a note in the PR body

## Where truth lives
- specs & PRDs → /docs/specs
- decisions → /docs/adr · wiki → {link}
- infra state → /infra (Terraform = source of truth)

## When unsure
Prefer reading the codebase over asking.
Ask only when two valid paths conflict.
```

The test: could a talented engineer with zero history here open this file and make a correct PR today? If yes, an agent can too.

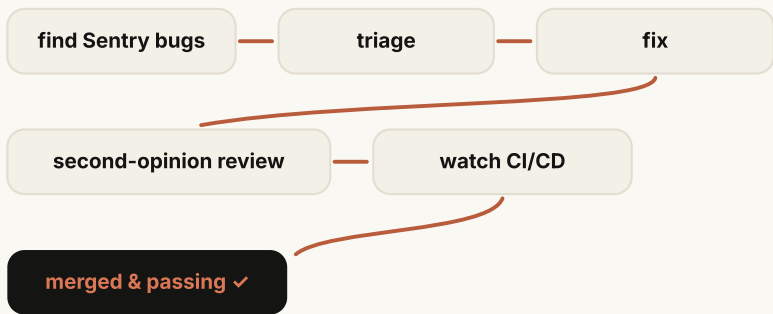
Let the agent gather its own context — and take action.

Anything the agent can use to proactively pull in information, infrastructure, and data — or to act inside your repository. Without tools, you're back to babysitting. [06:13]

TOOL	WHAT IT UNLOCKS FOR THE AGENT	THE MOMENT YOU FEEL IT
grep · web search	Orients itself in the codebase and the outside world without you narrating either.	It cites the file you forgot existed.
gh (GitHub CLI)	Reads old PRs and history, creates issues and PRs, watches checks — your repo's memory. [14:51]	It references a decision from a PR six months back.
Playwright MCP	Gives the model eyes : screenshot its own front-end work and compare against the mock.	It catches its own overlapping div before you do.
Sentry CLI	Views and triages production issues, then drives fixes end to end. [15:06]	A bug becomes a reviewed PR while you're at lunch.
gcloud (read-mostly)	Pulls logs, infra config, machine specs, deploy status, telemetry — real production context on demand. [15:38]	It answers "why did the deploy fail?" itself.
Terraform	Reads your infrastructure <i>as code</i> — current state, what changed, why, and when.	Infra questions stop reaching you at all.
Supabase / DB CLI	Inspects schema and platform state directly instead of asking you to describe it.	Schema questions disappear from your inbox.

// WORKED EXAMPLE — SENTRY - FIXES FROM THE VIDEO [15:06]

One custom command, invoked by name, runs the whole loop — because the agent has the Sentry CLI, `gh`, and CI access to do every step itself:



THE LITMUS TEST

"It just answered its own question — that's the point."

Every question an agent answers with a tool is a question that never interrupts you. That's what makes eight agents in parallel survivable.

My two favorite tools to expose: Terraform & gcloud.

Documenting your infrastructure to your agents is a huge unlock — and it only works if the permission structure makes free rein safe. [15:31]

// GCLOUD, READ-MOSTLY

I use gcloud in a mostly **read-only** context: logs, infra configuration, machine specs, failed deploys, telemetry and performance data. It gives the model an extraordinary amount of context about production — exactly when I'm rolling something out or triaging a real prod issue. [15:38]

// TERRAFORM = INFRA THE MODEL CAN READ

Infrastructure as code lays out everything in a standardized way the agent can consume: current state, what changed, why, and when. My drift-check habit: have the agent compare the live platform (Supabase / GCP / AWS CLIs) against the Terraform config and surface any diffs to apply back. [16:13] Production and config stop drifting apart — and the agent is the one watching.

▲ THE WARNING THAT MATTERS [27:28]

Set up permissions so agents have **free rein to read** — logging, config, state — but **no changes to core infrastructure without human review**. Never hand a headless agent your user-permissioned console access. The real guardrail is IAM (a viewer-role service account), with CLI allowlists on top.

// SCOPED PERMISSIONS TEMPLATE

```
.claude/settings.json · adapt to your stack

{
  "permissions": {
    "allow": [
      // read the world freely
      "Bash(gh pr view:*)", "Bash(gh pr checks:*)",
      "Bash(gh run view:*)", "Bash(gh search:*)",
      "Bash(gcloud logging read:*)",
      "Bash(gcloud run services describe:*)",
      "Bash(gcloud run services list:*)",
      "Bash(terraform plan:*)",
      "Bash(terraform state list:*)",
      "Bash(supabase inspect:*)"
    ],
    "deny": [
      // mutations wait for a human
      "Bash(terraform apply:*)",
      "Bash(terraform destroy:*)",
      "Bash(gcloud run deploy:*)",
      "Bash(gcloud iam:*)",
      "Bash(rm -rf:*)"
    ]
  }
}
```

Pair with a **read-only service account** (e.g. `roles/viewer` + `roles/logging.viewer`) so the boundary holds even if a command slips past the allowlist. Set the IAM structure up *now* — before next quarter's capability jump forces a panic rework.

overheard →

On a recent *Cheeky Pint* (John Collison's podcast), the gcloud CLI got the same rave. If you haven't let agents help build infrastructure — safely, through Terraform and scoped cloud CLIs — it's an absolute game changer. [28:06]

Package your judgment, not just your knowledge.

SOPs encode the considerations, judgments, and bits of taste your team applies — so the model is forced through the same steps you'd take. That's the difference between "a PR I trust" and "a PR I audit line by line." [06:34]

// THE OFF-THE-SHELF START: GSTACK [06:56]

gstack packages Y Combinator's accumulated best practices — Garry Tan and team's office hours, UX/UI coaching, security auditing — into invokable workflows (~109k GitHub stars at recording). The skills can be bloated at times, but they force the model to think through a stack of perspectives before writing a line of code:

/office-hours	Initial intake — your company, product, and goal, YC-style. Run this first.
/plan-ceo-review	The product lens: do customers want this? Is it scoped down to the right size?
/plan-eng-review	Technical architecture and the main engineering considerations. My most-used, by far.
/plan-design-review	UI edge cases plus the polish and niceties that make an elegant interface.
/devex	Developer-experience pass — a stand-in for the <i>agent's</i> experience too.
/review + adversarial	Post-build review; from Claude Code it can invoke a CLI Codex to challenge the work (and vice versa). [10:32]

THE INTERVIEW PATTERN [09:20]

Let the skills **interview you**. They surface batches of questions; you answer, let it run, come back, answer more. The model does 80% of the thinking against industry best practice — you jump in at the end and layer your expertise on top. Half the time the recommended answer is right; I'm picky about the rest.

// TASK DISCIPLINE: BEADS [23:59]

Steve Yegge's open-source task system keeps agents honest on long horizons — issue by issue, dependency graph and all, like a Linear/Jira built for agents. A markdown plan gets skimmed as the context window fills; a task database doesn't. Claude Code's native Tasks feature was modeled on Beads (they credited Yegge in the release).

Run `bv` — the bead viewer — to see the task hierarchy your agents are working; an easy check that long-running sessions **did what they claim they did**, instead of hallucinating half the checkmarks. If I sound bitter, it's because I've been burned. [25:20]

Then build your own. The highest-leverage move is wrapping gstack's primitives in skills that encode *your* process — which is exactly what `review-and-push` does, next page.

One command from PR to green CI — audited, fixed, pushed.

My actual skill file, excerpted. It composes gstack's `/review` and `/ship` with an explicit Codex adversarial challenge, PR-comment triage, and a CI watch loop. Invoke it with a PR number and walk away. [10:56]

// THE EIGHT PHASES

- 0 **Preflight**
branch · tree · PR · auth checks
- 1 **PR setup**
create one via `/ship` if missing
- 2 **gstack /review**
auto-fixes the clear stuff
- 3 **Codex challenge**
adversarial: edge cases & security
- 4 **Comment fetch**
CodeRabbit + humans + bots, deduped
- 5 **Triage manifest**
every finding classified — the audit trail
- 6 **Apply fixes**
atomic commits per logical group
- 7 **Push & CI watch**
≤2 auto-fix attempts · 30-min cap
- 8 **Final report**
changed · skipped · CI state, scannable

Why the second Codex pass when `/review` includes one? Different framing, deduped output — and leveraging the **diversity of training data and post-training** across models is my favorite workflow of all. Different RLHF, different blind spots. [11:44]

// THE PART WORTH STEALING: TRIAGE RULES REAL FILE

```
review-and-push/SKILL.md · excerpt, condensed

### Confidence rules
Human reviewer → start HIGH. Downgrade only if the
comment misreads the code (verify by re-reading
the code, not just the diff). Never start low.
gstack /review → medium-high; upgrade to HIGH when
the Codex challenge independently agrees.
Codex challenge → medium; HIGH with a concrete repro.
CodeRabbit → start LOW. Read the code and form
your own opinion; upgrade only on independent
agreement — never on the bot's say-so.

### Decision rules (lean toward act)
act → HIGH confidence, or MEDIUM and the
category is correctness / security / tests /
error-handling / data-integrity.
skip-with-reply → finding is wrong or out of scope;
reply with a substantive reason. Never "won't fix."
skip-silent → style nits and naming opinions
where the code is correct.
Disagreeing with a reviewer is allowed — but the
rationale must land in the manifest. No silent skips.

### Non-negotiables
- Never amend or force-push — every fix bisectable
- Never bypass hooks (--no-verify)
- Never push to main — branch off first
- Never reply to a comment without reading the
surrounding code — bots and humans both misread
- The triage manifest is mandatory — write it
before fixing anything. It's the contract.
```

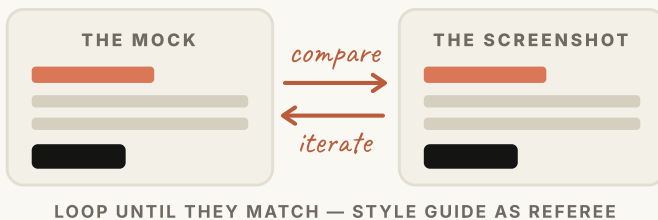
The manifest is what makes hands-off safe: every finding, every decision, every disagreement — written down where you can audit it after the fact. Delegation without a paper trail is just hoping.

Show the model what finished looks like.

You need the kickoff context — and then clear examples of "done," plus what still has to change to get there. That's what lets the loop run without you. [14:22]

// VISUAL VERIFICATION — GIVE IT EYES [13:18]

Put the UI mock in the spec. Then have the model use **Playwright to screenshot its own work** and compare against that mock — spotting overlap, critiquing its own design, iterating until they match, with your style guide as the referee.



THE DEAD-CODE LESSON [14:22]

If you don't *call out* the dead code with a tool, I guarantee the model will not holistically audit and remove it. What isn't checked deterministically doesn't reliably happen. Anything that **can** be deterministic **should** be.

// THE DETERMINISTIC STACK [13:44]

Anything that runs in a CLI and audits code belongs in the loop — the agent runs these itself and reacts to the output:

- ☐ **Linters & formatters** — ESLint and friends; zero-argument style enforcement
- ☐ **Unit tests** — models are incredible at writing them; there's no reason not to run near-TDD, especially on the back end
- ☐ **Static analysis & type checks** — cheap, instant, unambiguous
- ☐ **Dead-code / unused-variable audits** — e.g. knip or ts-prune for TypeScript
- ☐ **CI/CD gates** — the plethora of tests and checks that run on every push, and that agents can watch and react to

100→5

QUESTIONS YOU ANSWER
PER LONG-RUNNING TASK

8–10

AGENTS IN PARALLEL
ONCE QUESTIONS VANISH

That 100→5 number is the whole game. [17:02] Context-switching between eight or ten agents destroys short-term memory; the only recovery is agents that come back *already* QA'd. Set and forget — then read the report.

`$ rm -rf ~`

The cautionary tale: without verification you start clicking *next, next, next* on permission prompts without reading — right up until you've deleted something you can't get back. Guardrails are what make speed safe. [17:38]

A production analytics platform, end to end, within a week.

Pub/Sub, Cloud Run, BigQuery — full production scale, built for spiky live-event traffic. Every pillar was in place before the work started. [21:17]



The kickoff was nothing fancy: UI mocks and rough sketches, several meeting transcripts, and CEO notes, thrown into the model as one PRD-style brief — plus an HTML page from **Claude Design** to pin down the aesthetic and feature set. [21:17] The infrastructure behind it aggregates spiky live-event traffic (NFL- and FIFA-scale spikes), with PDF export and advanced sharing on top.

[22:05]

A workflow note that surprises people: the "engineering manager" session delegating to sub-agents means you talk to *one* agent about day-to-day questions, and it manages the rest — more on that next page. [26:07]

6h

OVERNIGHT
CODEX RUN,
FULLY
AUTONOMOUS

~1wk

CEO INBOUND →
PRODUCTION,
END TO END

50x+

THE REALM
THIS PUTS
YOU IN — DAYS,
NOT WEEKS

SIDEBAR — CLAUDE DESIGN AS A CONTEXT MACHINE [22:47]

Build an entire design system — iconography, white space, typography — then export real design files that anchor the agents' UI work. (Same process produced my personal site's new direction: fun, creative, entrepreneurial — *not* cyberpunk AI.) Image generation in ChatGPT, or your own mocks, work as inputs too.

Stop being the engineer. Manage the manager.

Ask Claude Code to create a team of other Claude Codes and manage them — five PRs, or twenty, for a set project. You go up one more level of abstraction. [26:07]



engineer → eng manager → CTO of agents

// WHY IT WORKS

The sub-agents have enough context to go end to end; the manager has enough context to manage them. **Both facts come from the same investment** — the spec and the orchestration layer. Skimp there and the whole tree collapses into you answering questions again. [26:27]

With Fable and Astra class models, design the workflow so the session can delegate to specialists for different areas of the code, each invoking its own sub-skills. My review-and-push workflow runs its adversarial pass alongside the core review. Agent nesting depends on the tools and runtime you use. [19:09]

// THE EMERGENT PROPERTY: PROACTIVE AGENTS

[20:28]

Once the layer is rich enough, models start **proactively suggesting ideas** — they know your company, your product, your deterministic needs, and they have the tools to go research. Point one at your issue tracker and it will: pick up the bug → write a test → fix the regression in your styling conventions → run it through QA → open a PR.

THE REFRAME

Constraints creatively free the models. Just enough guardrail lets them discern good ideas from bad, spot the obvious issues, and package work so it would actually get merged. [20:59]

Three patterns that didn't survive the cut.

Straight from my planning notes for this video — trimmed for runtime, too useful to leave out.

Consider this the director's cut. FIELD NOTES

a

AUTO-ACCEPT, CLASSIFIED

Bouncing between 6–10 agents' permission prompts is brutal — and blanket "accept everything" is how repos get deleted. The pattern: a **second model acts as a permissions classifier**, deciding which requests are routine (auto-approve) and which genuinely need you.

It replaces sandboxing pain and "dangerously accept all" risk with a judgment layer — and it's the practical unlock for running agents in parallel.

the unlock for parallelism

b

ULTRA REVIEW

Anthropic's cloud-based, **team-of-agents code review** — multiple reviewer agents working a change over in parallel. More diligent than any review process I've used, human or automated.

The discipline: **reserve it for high-stakes architectural changes**, not every PR. Day-to-day, the review-and-push loop on p.9 is the workhorse; this is the heavy artillery.

heavy artillery, used sparingly

c

ROUTINES → SELF-HEALING

Routines trigger Claude Code automatically on events. Mine: a **Sentry webhook fires → the agent investigates → a PR appears** — with no human in the loop until review. It's the sentry-fixes workflow from p.6, running itself.

This is the glimpse of where it all points: **software that notices its own problems and drafts its own fixes** while you review PRs and staging sites.

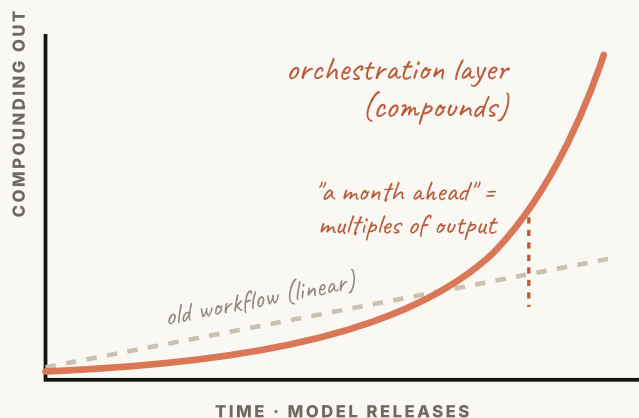
self-healing software, today

WHY THEY COMPOUND

Each pattern removes a category of interruption: (a) removes permission prompts, (b) removes review anxiety on big changes, (c) removes the triage queue itself. Stack them on the four pillars and you go from one agent running well to **ten agents running in parallel** — an AI software factory.

Why a week ahead is worth more than a week.

The strategic frame that makes the investment obvious — and the six moves to take away. [28:33]



Picture the graph: x-axis is time, y-axis is your startup's compounding output through AI. Right now, being a week or a month ahead is an **exponentially increasing** advantage — in distribution, and especially in codegen infrastructure. A few more automations than the next team compounds at a ridiculous rate. The good news: **a little further ahead on the curve is all you need.** [28:53]

// THE ONE INVESTMENT THAT NEVER EXPIRES

[29:37]

The single best thing I invest in — the one thing I don't worry about becoming obsolete — is **documenting our company's context** and building the systems that pull it into one orchestration layer. However good the models get, they will *never* have your workflows, tooling, and verification unless you outline them explicitly.

BUILD FOR THE MODELS YOU'LL HAVE, NOT THE ONES YOU HAVE

Build expecting the models to get fantastic and the harnesses to get much better. Invest in what won't be disrupted by that — the things that get **leveraged and amplified** instead. [30:04]

*The mindset that wins this era isn't cost-cutting. It's **abundance** — invest in the layer that extracts more from every token as the models keep getting better.*

// THE SIX TAKEAWAYS

1. Orchestration layer first, model second.

The layer is the ceiling; raise it before chasing releases.

2. Test models with end-to-end tasks.

Single prompts can't reveal long-horizon capability.

3. Run agents in parallel, safely.

Classified auto-accept + scoped permissions, never blanket yes.

4. Layer in second-opinion review.

Cross-model challenge (Codex ↔ Claude) + heavy review when stakes are high.

5. Document SOPs and skills, not just code.

Package your judgment so agents (and teammates) follow the same process.

6. Treat infra as scoped, versioned context.

Terraform for truth, read-only cloud access, IAM set up *now*.

One pillar per week. Ship proof each Friday.

A suggested sequence for building the layer without pausing feature work — each week ends with a concrete artifact and a test that tells you it's working.

WEEK	BUILD	YOU KNOW IT'S WORKING WHEN...
WK 1 CONTEXT	Write the onboarding layer. Draft <code>CLAUDE.md</code> from the p.5 skeleton (scope in/out, design language, quality bar, where truth lives). Pipe in the exhaust: link your wiki, drop in the latest planning transcript, point at adjacent repos.	A fresh session makes a correct, convention-following change with zero clarifying questions.
WK 2 TOOLS	Wire the hands and eyes. Install <code>gh</code> , Sentry CLI, Playwright MCP. Create the read-only cloud service account; import infra into Terraform if it isn't already. Add the scoped allow/deny list from p.7.	You ask "why did staging break?" and the agent answers from logs — without you opening a dashboard.
WK 3 WORKFLOWS	Install the process. Add gstack; run <code>/office-hours</code> to seed company intake. Adapt <code>review-and-push</code> (p.9) to your repo — start with the triage manifest and the non-negotiables. Adopt Beads or native Tasks for anything multi-hour.	One real PR goes intake → review → adversarial challenge → green CI through a single command.
WK 4 VERIFY + SCALE	Close the loop, then parallelize. Get linters, unit tests, type checks, and a dead-code audit into CI. Stand up the mock-vs-screenshot compare. Then: run <i>two</i> agents on separate PRs; grow toward 6–10 only as the question count stays near zero. Finish with an ambitious overnight run.	You come back in the morning to reviewed, CI-green work — and read a report instead of a backlog of questions.

THE ORDER MATTERS

Context before tools (tools without context gather noise).
Tools before workflows (SOPs assume the agent can act).
Workflows before scale (parallelism multiplies whatever process exists — good or bad). Don't start at week 4.

Measure one number all month:
questions answered per completed task.

watch it fall from 100 toward 5

The stack, linked.

Everything named in the video and this playbook — grouped by pillar, so you can wire it in the same order you build.

WORKFLOWS & SOPS

gstack	YC workflow pack (Garry Tan) — office hours, plan reviews, /review, /ship github.com/garrytan/gstack
Beads	Agent task discipline (Steve Yegge); <code>bv</code> is its task viewer github.com/steveyegge/beads
Claude Design	Design systems & UI files as agent context anthropic.com/news/claude-design-anthropic-labs

TOOLS TO EXPOSE

gh CLI	GitHub from the terminal — cli.github.com
Playwright MCP	Browser eyes for the model — github.com/microsoft/playwright-mcp
Sentry CLI	Issue triage & investigation — docs.sentry.io/product/cli
gcloud · Terraform	Infra context, read-mostly — cloud.google.com/cli · developer.hashicorp.com/terraform
ESLint · knip	Lint + dead-code audits — eslint.org · knip.dev

CLAUDE CODE FEATURES

Code Review	Team-of-agents "Ultra Review" — code.claude.com/docs/en/code-review
Routines	Event-triggered agents (the Sentry webhook flow) — code.claude.com/docs/en/routines
Tasks	Native task graph, modeled on Beads

WORTH YOUR EARS

OpenAI Podcast	Greg Brockman on the AGI-pilled Codex vision (Ep. 6, "Codex and the future of coding with AI")
Cheeky Pint	John Collison's podcast — where the gcloud CLI got its rave — cheekypint.transistor.fm

KEEP GOING

This playbook pairs with the video — and the next ones. I share field notes on power workflows for Claude Code, Codex, and second brains as we build.

Subscribe	youtube.com/@PatrickOakleyEllis
Learn more	patrickellis.io
Next video	The design deep dive — getting the best UI out of Claude Code & Codex

Build expecting the models to get fantastic — and invest in the things that get amplified, not disrupted. Every release should make you **faster, not more anxious**.

— Patrick